

Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação
Computação de Alto Desempenho — SSC0903

Primeiro Trabalho Prático (TB1)

Simulação paralela da propagação direcional de incêndio com
zonas de contenção

Integrantes

Felipe de Castro Azambuja — 14675437
Gabriel de Andrade Abreu — 14571362
João Pedro Viguini Tolentino Taufner Correa — 14675503
Matheus Paiva Angarola — 12560982
Pietra Gullo Salgado Chaves — 14603822

São Carlos, 2026

Sumário

1	Introdução	1
1.1	Objetivos	1
1.2	Resumo da abordagem	1
2	Solução Sequencial	1
2.1	Estruturas e preparação	1
2.2	Passo da simulação	2
2.3	Ativação das zonas de contenção	2
2.4	Cálculo de estatísticas e <i>checksum</i>	2
3	Solução Paralela com OpenMP	2
3.1	Região paralela persistente	3
3.2	Paralelização da ativação das zonas	3
3.3	Paralelização da atualização da matriz	3
3.4	Reduções para contadores	3
3.5	Uso de SIMD e diagnóstico do compilador	3
3.6	Cláusulas de escopo e <code>default(none)</code>	4
3.7	Troca segura das matrizes e condição de parada	4
3.8	Diagrama da solução	4
3.9	Comparação de <i>schedules</i>	5
4	Validação	5
4.1	Metodologia de comparação	5
4.2	Casos de teste	6
4.3	Determinismo em relação ao número de threads	6
5	Ambiente e Metodologia Experimental	6
5.1	Hardware e software	6
5.2	Parâmetros das entradas de teste	7
5.3	Metodologia de medição	7
6	Resultados	8
6.1	Visualização do comportamento das entradas	8
6.2	Tempo, <i>speedup</i> e eficiência	10
6.3	Comparação de <i>schedules</i>	12
6.4	Efeito de <code>-O3</code> e das posições SIMD	13
7	Análise dos Resultados	15
7.1	Escalabilidade e gargalos	15
7.2	Fração serial efetiva (Karp–Flatt)	15
7.3	Justificativa do <i>schedule</i> escolhido	15
7.4	Interpretação da otimização e do SIMD	16
8	Conclusão	16

1. Introdução

Este trabalho implementa uma simulação determinística da propagação de incêndio em uma matriz retangular. Foram desenvolvidas uma versão sequencial em C e uma versão paralela com OpenMP. Ambas consideram cobertura, umidade, vento, focos iniciais, tempo de queima e zonas de contenção. O objetivo da paralelização é reduzir o tempo do núcleo da simulação sem modificar o resultado.

As duas versões compartilham as mesmas regras e estruturas. A comparação experimental verifica a correção, o tempo de execução, o *speedup*, a eficiência e o efeito de diferentes *schedules*.

1.1 Objetivos

Os objetivos são implementar corretamente as regras da simulação, garantir resultados idênticos entre as duas versões e avaliar a paralelização por meio de tempo, *speedup*, eficiência e comparação de *schedules*.

1.2 Resumo da abordagem

A mesma representação linear e a mesma lógica de atualização são usadas nos dois programas. A versão sequencial serve como referência, enquanto a versão OpenMP divide os laços sobre as células entre as threads e sincroniza as estatísticas e a troca de matrizes ao final de cada passo.

2. Solução Sequencial

Esta seção apresenta a organização da versão sequencial, usada como referência de correção. São descritos o armazenamento das matrizes, as regras de propagação, a ativação das contenções e o cálculo das estatísticas finais.

2.1 Estruturas e preparação

As matrizes são armazenadas linearmente pelo índice

$$indice = linha \times C + coluna.$$

Para cada célula são mantidos cobertura, umidade, estado e tempo de queima. A simulação utiliza pares de vetores para estado e tempo: `estado_atual/proximo_estado` e `tempo_atual/proximo_tempo`. Essa organização permite calcular o próximo passo exclusivamente a partir do passo atual.

A cobertura e a umidade são geradas sequencialmente, em ordem linear, com `rand_r`. Em seguida são aplicados os focos iniciais e construído o vetor `ativacao`.

Quando zonas se sobrepõem, o vetor guarda o menor passo de ativação.

2.2 Passo da simulação

Em cada passo, o programa:

1. ativa as contenções programadas sobre células intactas;
2. calcula o próximo estado e o próximo tempo de todas as células;
3. conta novas ignições e células que continuarão em chamas;
4. troca os pares de matrizes;
5. encerra caso não existam células em chamas ou tenham sido executados P passos.

Para uma célula intacta, são examinados os oito vizinhos de Moore. Cada vizinho em chamas contribui com peso básico 10, se ortogonal, ou 7, se diagonal. Considerando o alinhamento A com o vento, o peso é

$$P_v = \max(1, P_{basico} + intensidade \times A).$$

O potencial de ignição é calculado com aritmética inteira:

$$I = \left\lfloor \frac{S \times fator_{combustivel} \times (100 - umidade)}{100} \right\rfloor.$$

A célula entra em chamas quando $I \geq LIMIAR$. O tempo de queima é 2 para vegetação rasteira e 4 para floresta.

2.3 Ativação das zonas de contenção

No início do passo p , células intactas cujo valor em `ativacao` seja igual a p passam ao estado de contenção. Células não combustíveis, em chamas, queimadas ou já protegidas não são alteradas. Uma zona atua somente no passo programado.

2.4 Cálculo de estatísticas e *checksum*

Ao final são calculados os estados, o total e o pico de ignições, os percentuais queimado e protegido e o *checksum*. O *checksum* é sequencial e fica fora do trecho cronometrado.

3. Solução Paralela com OpenMP

Esta seção descreve como o núcleo sequencial foi paralelizado com OpenMP sem alterar as regras da simulação. Também são discutidos o compartilhamento de dados, as reduções, a sincronização e os últimos critérios de implementação.

3.1 Região paralela persistente

A versão paralela utiliza uma única região persistente com T threads e `default(none)`. A leitura da entrada e a geração pseudoaleatória da floresta ficam fora do trecho cronometrado e permanecem sequenciais.

3.2 Paralelização da ativação das zonas

A varredura do mapa de ativação é distribuída com `omp for` e `collapse(2)`. Cada iteração trata uma célula diferente, portanto as threads não escrevem na mesma posição.

3.3 Paralelização da atualização da matriz

Dentro de cada passo, três operações sobre a matriz são distribuídas com `omp for`: ativação das contenções, atualização das células e cálculo das estatísticas. Como cada iteração de atualização escreve em uma posição distinta dos vetores de próximo estado e próximo tempo, não há disputa de escrita entre threads. A contagem das novas ignições e das células em chamas combina `omp for simd` com redução. Após o último passo, a mesma equipe persistente distribui a contagem dos cinco estados finais com outra construção `omp for simd`.

3.4 Reduções para contadores

Reduções OpenMP calculam as novas ignições e as células em chamas em cada passo. Uma segunda redução, ainda dentro da região persistente, acumula as quantidades finais de células não combustíveis, intactas, em chamas, queimadas e de contenção. Isso dispensa `critical` e `atomic` nos laços. Uma única thread atualiza o pico, acumula o total de ignições e troca os ponteiros com `single`. As barreiras implícitas asseguram que todas as threads terminem uma etapa antes da seguinte e observem a mesma condição de parada.

3.5 Uso de SIMD e diagnóstico do compilador

As duas reduções lineares usam `simd`: a contagem de novas ignições e células em chamas a cada passo e a contagem dos cinco estados finais. O índice percorre diretamente o vetor de 0 a $LC - 1$, com acessos contíguos e sem dependência entre iterações. Expressões como `estado == 2` não criam, por si, uma quebra de fluxo problemática: em C elas produzem 0 ou 1 e podem ser implementadas como máscaras vetoriais. O requisito decisivo é que iterações distintas não disputem posições de memória e que os acumuladores sejam declarados por redução.

A diretiva `simd` autoriza a vetorização, mas não obriga o compilador a emitir instruções vetoriais. No GCC 5.3.1 usado nas medições, o relatório de otimizações confirmou explicitamente a vetorização da contagem final; a redução linear por passo

não recebeu confirmação explícita nesse compilador, o que não afeta a correção nem o resultado.

Também foram medidas versões com `simd` na ativação das contenções, no laço principal de atualização e nos dois locais simultaneamente. Todas permaneceram funcionalmente corretas, mas ficaram mais lentas. O laço principal contém bordas, vizinhança de Moore, condições e chamada auxiliar; uma vetorização eficaz exigiria reorganização algorítmica própria. Por isso, a versão final mantém SIMD somente nas duas reduções lineares cuja forma é simples e auditável.

3.6 Cláusulas de escopo e `default(none)`

As dimensões, parâmetros, matrizes e contadores são compartilhados. O passo é inicializado com `firstprivate`, e os índices dos laços são privados por definição. O uso de `default(none)` torna essa classificação explícita e reduz o risco de compartilhamento incorreto.

3.7 Troca segura das matrizes e condição de parada

As matrizes são alternadas por troca de ponteiros (*double buffering*), evitando cópias completas ao final de cada passo. A troca e a atualização de `passos_executados` ocorrem em uma região `single`. A barreira implícita ao final dessa região garante que nenhuma thread inicie o passo seguinte com ponteiros antigos. A condição de parada utiliza o contador compartilhado produzido pela redução anterior.

3.8 Diagrama da solução

A Figura 1 apresenta o fluxo efetivo da versão OpenMP, incluindo o limite do cronômetro, a equipe persistente, os buffers compartilhados e a contagem final. As barreiras indicadas são implícitas ao final das regiões de compartilhamento de trabalho e das regiões `single`.

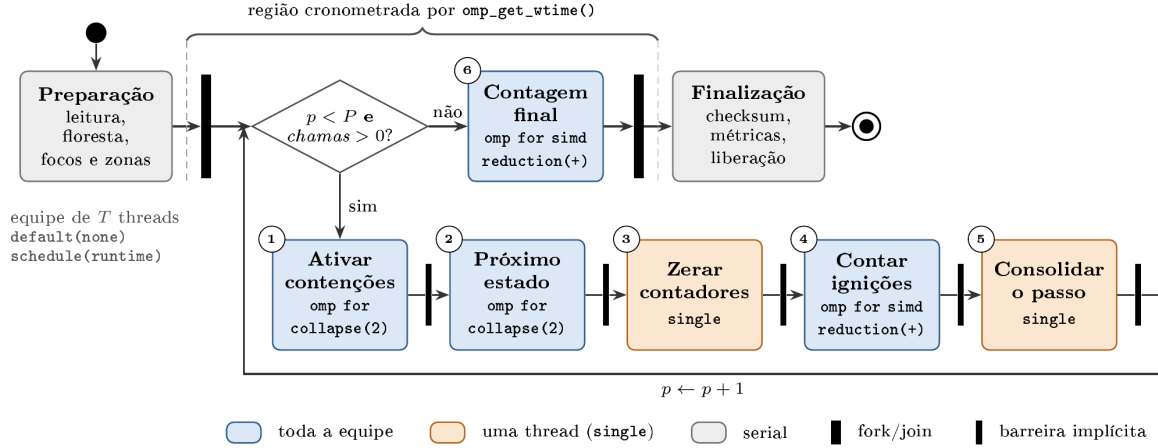


Figura 1: Fluxo da versão OpenMP. Uma única equipe de T threads é criada uma vez e percorre todos os passos: a cada passo são ativadas as contenções, calculado o próximo estado de cada célula, zerados os contadores por uma thread, somadas as novas ignições por redução e consolidado o passo (troca das matrizes) por uma thread; ao fim dos passos, os cinco estados finais são contados. O cronômetro cobre do início dessa região até o fim da contagem final, deixando de fora leitura, alocação, geração da floresta, *checksum* e impressão. As barras verticais representam barreiras implícitas.

3.9 Comparação de *schedules*

Foram avaliados *static*, *dynamic* e *guided*, com diferentes tamanhos de bloco. Os resultados e a justificativa da configuração mantida no programa final são apresentados na Seção 6.3.

4. Validação

Esta seção verifica se as versões sequencial e paralela produzem exatamente o mesmo resultado. A comparação considera as saídas completas, os índices de verificação e diferentes quantidades de threads.

4.1 Metodologia de comparação

Cada execução paralela foi comparada automaticamente com uma execução sequencial da mesma entrada, ignorando somente o campo de tempo; qualquer diferença em outro campo é tratada como falha. Também foram verificadas, após cada execução, a conservação do número total de células e a igualdade entre as células atingidas pelo fogo e os focos iniciais mais as novas ignições.

Além das três entradas oficiais, foi usada uma implementação de referência independente para 80 grades pequenas, incluindo dimensões degeneradas, $P = 0$, diferentes ventos, limiares, sementes, focos e zonas sobrepostas. Cada caso foi comparado com a versão sequencial e com a versão OpenMP usando 1, 2, 3, 4 e 8 threads e três *schedules*,

totalizando 1.280 execuções válidas. Foram verificadas ainda 36 execuções de entradas inválidas.

4.2 Casos de teste

Os resultados resumidos na Tabela 1 confirmam a equivalência das duas implementações nas três cargas avaliadas.

Tabela 1: Validação das versões sequencial e paralela.

Entrada	Dimensão	Passos	Threads	<i>Checksum</i>	Resultado
Pequena	400×500	80	4	17073017104646724864	Idêntico
Média	1200×1500	100	8	17187382406529706254	Idêntico
Grande	2500×2500	100	8	14825089465781038537	Idêntico

4.3 Determinismo em relação ao número de threads

Nas grades pequenas, a saída paralela coincidiu com a referência independente e com a versão sequencial para 1, 2, 3, 4 e 8 threads, sob `static`, `dynamic` e `guided`. Nas entradas oficiais, foram testadas 1, 2, 4, 8 e 16 threads. Em todos os casos, os campos funcionais permaneceram idênticos; somente o tempo variou.

5. Ambiente e Metodologia Experimental

Esta seção caracteriza a plataforma de testes e o procedimento usado para obter as medições. São apresentados o hardware, o software, as cargas, as repetições e as métricas adotadas.

5.1 Hardware e software

A Tabela 2 registra a configuração do nó `ha101`, no qual todas as medições apresentadas neste relatório foram executadas. A compilação usou as flags `-Wall -Wextra -O3 -fopenmp`. Aqui, `-O2` e `-O3` são níveis de otimização do compilador GCC: `-O3` habilita transformações mais agressivas que `-O2` — como vetorização adicional e desenrolamento de laços — sem alterar a semântica do programa.

Tabela 2: Ambiente utilizado nos experimentos.

Item	Especificação
Processador	Intel Core i7-4790, 3,60 GHz
Núcleos / processadores lógicos	4 / 8
Memória RAM	31 GiB
Sistema operacional	Ubuntu 24.04.4 LTS, Linux 5.15.75, x86_64
Compilador	GCC 5.3.1
Flags	-Wall -Wextra -O3 -fopenmp

5.2 Parâmetros das entradas de teste

As cargas usam matrizes de 400×500 , 1200×1500 e 2500×2500 , com 80, 100 e 100 passos máximos, respectivamente. Elas incluem diferentes sementes, direções e intensidades de vento, focos iniciais e zonas de contenção, permitindo avaliar o algoritmo em escalas distintas.

5.3 Metodologia de medição

As medições foram executadas de forma serial, uma configuração por vez, no mesmo nó e com os mesmos binários. Cada configuração teve uma execução de aquecimento, descartada, seguida de cinco execuções medidas. A ordem das configurações foi embaralhada a cada repetição para reduzir viés de ordem. Os gráficos apresentam a média e, para os tempos, barras com intervalo de confiança de 95% calculado com a distribuição t de Student.

Todas as medições foram automatizadas, sem comandos manuais por configuração, e os dados brutos, o ambiente e os resumos foram preservados, de modo que as tabelas e figuras podem ser reproduzidas a partir dos mesmos valores. Ao todo, foram 132 execuções de desempenho e 1.316 execuções de validação funcional.

O tempo foi medido por `omp_get_wtime()` somente no núcleo da simulação: ativação, propagação, estatísticas, troca de matrizes e condição de parada. Leitura, alocação, inicialização, *checksum*, percentuais e impressão ficaram fora da medição. O ambiente definiu `OMP_DYNAMIC=FALSE`, `OMP_PROC_BIND=FALSE` e `OMP_WAIT_POLICY=PASSIVE` em todas as medições.

Foram utilizadas as entradas pequena, média e grande fornecidas no projeto. Os testes de escalabilidade empregaram 1, 2, 4, 8 e 16 threads, todos com o escalonamento `static` — o padrão do programa quando `OMP_SCHEDULE` não é definida. A comparação entre escalonamentos é tratada à parte na Seção 6.3. Assim, 4 threads correspondem aos núcleos físicos, 8 utilizam os dois processadores lógicos de cada núcleo e 16 caracterizam a sobresubscrição. O *speedup* e a eficiência foram calculados por

$$S(T) = \frac{t_{seq}}{t_{omp}(T)} \quad \text{e} \quad E(T) = \frac{S(T)}{T}.$$

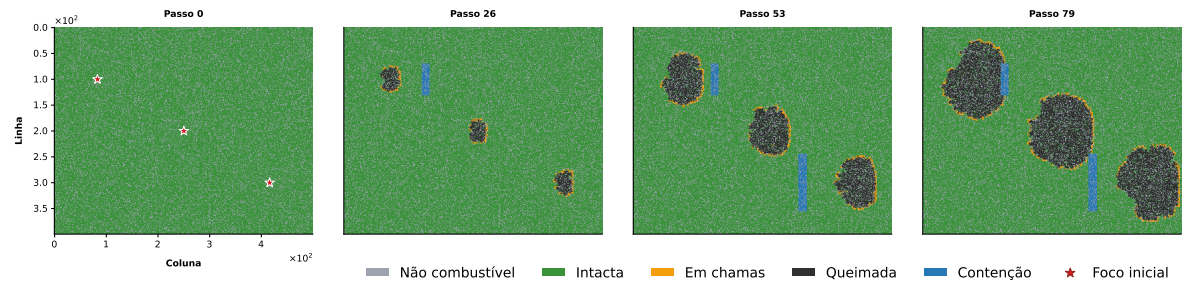
6. Resultados

Esta seção reúne os resultados funcionais e de desempenho. Primeiro é mostrado o comportamento da simulação e, em seguida, são comparados tempo, aceleração, eficiência e *schedules*.

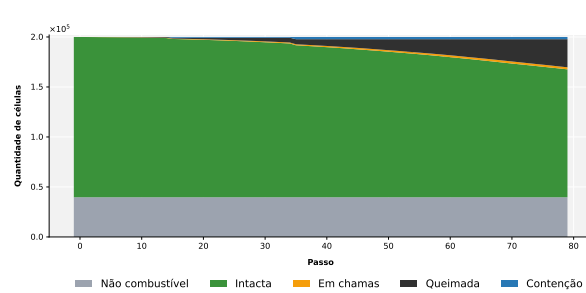
6.1 Visualização do comportamento das entradas

Para cada carga foram observados três aspectos: a distribuição espacial dos estados, a quantidade de células em cada estado e as novas ignições por passo. As figuras foram produzidas por um visualizador independente que reproduz as regras da simulação e foi conferido contra as métricas e os *checksums* dos programas em C. Como o modelo é determinístico e as saídas funcionais das versões sequencial e OpenMP são idênticas, uma única visualização representa ambas; a diferença entre elas é avaliada pelo tempo.

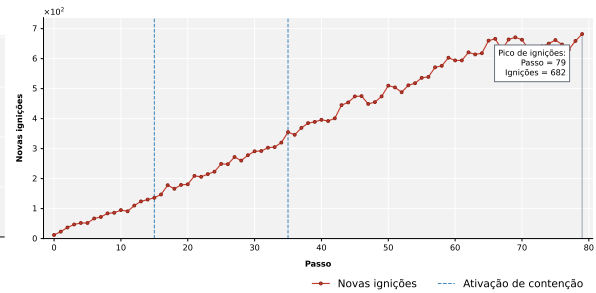
Entrada pequena



(a) Distribuição espacial dos estados.



(b) Evolução dos estados.

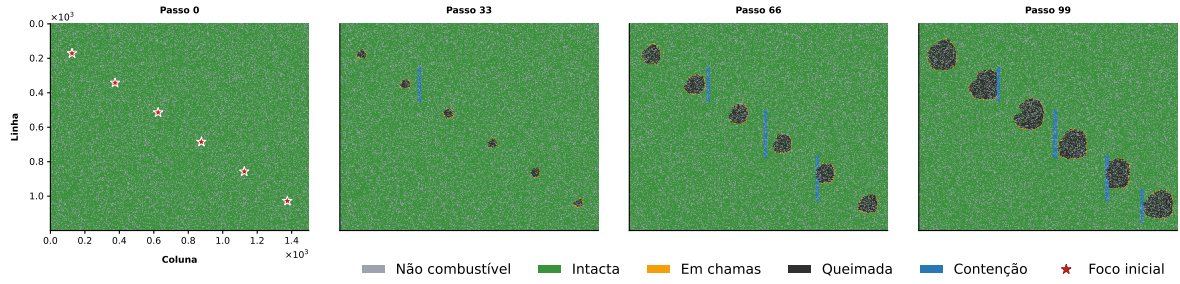


(c) Novas ignições por passo.

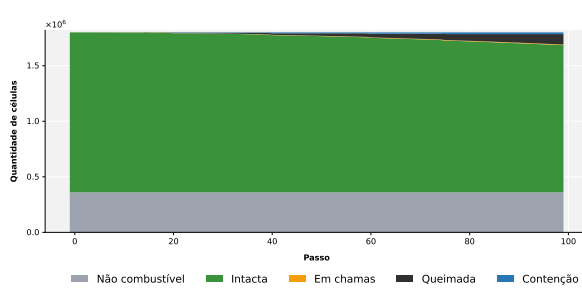
Figura 2: Entrada pequena (400×500 , 80 passos): (a) distribuição espacial dos estados em instantes selecionados; (b) evolução da contagem de células por estado; (c) novas ignições a cada passo.

Na Figura 2, as três frentes permanecem separadas durante os 80 passos. A distribuição espacial da Figura 2a mostra o bloqueio parcial causado pelas contenções dos passos 15 e 35; a Figura 2b mostra o crescimento gradual da área queimada, e a Figura 2c registra o pico de 682 ignições no passo 79.

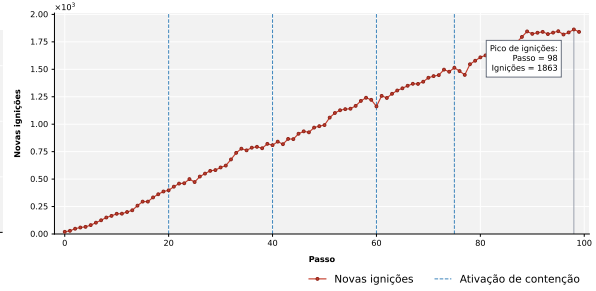
Entrada média



(a) Distribuição espacial dos estados.



(b) Evolução dos estados.

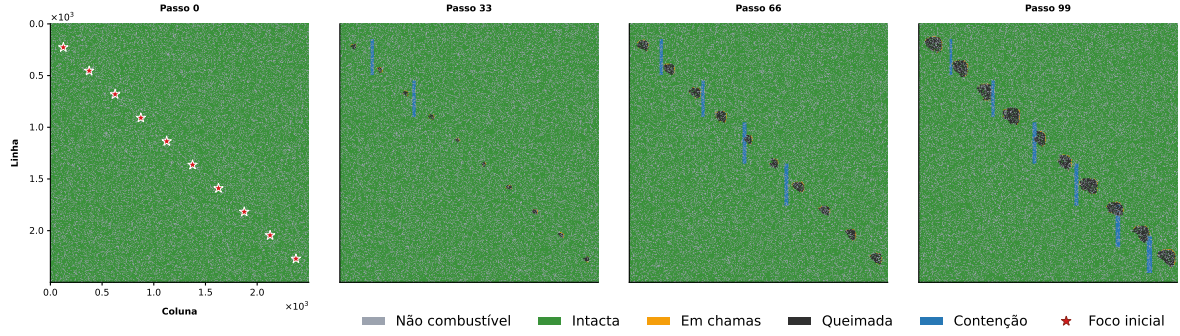


(c) Novas ignições por passo.

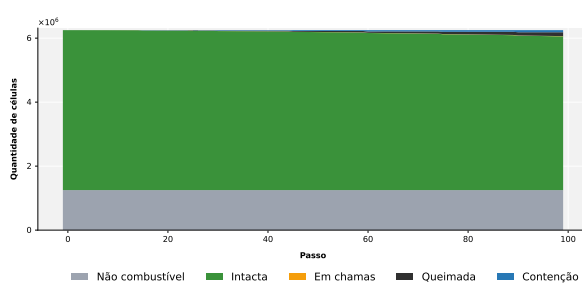
Figura 3: Entrada média (1200×1500 , 100 passos): (a) distribuição espacial dos estados em instantes selecionados; (b) evolução da contagem de células por estado; (c) novas ignições a cada passo.

Na Figura 3, os seis focos originam frentes maiores e as quatro zonas de contenção limitam regiões distintas. A Figura 3a evidencia essa distribuição; a Figura 3b indica que ainda há células em chamas no limite de 100 passos, e a Figura 3c mostra o pico de 1.863 ignições no passo 98.

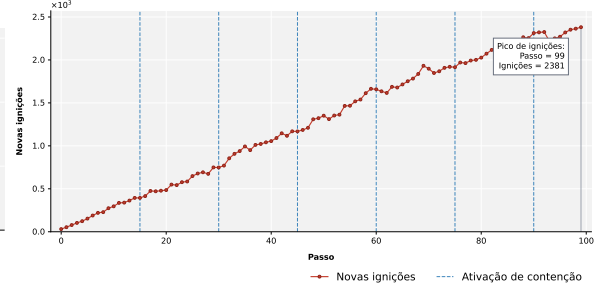
Entrada grande



(a) Distribuição espacial dos estados.



(b) Evolução dos estados.



(c) Novas ignições por passo.

Figura 4: Entrada grande (2500×2500 , 100 passos): (a) distribuição espacial dos estados em instantes selecionados; (b) evolução da contagem de células por estado; (c) novas ignições a cada passo.

Na Figura 4, os dez focos distribuem a propagação por uma área maior. As seis barreiras são visíveis na Figura 4a; a transição dos estados aparece na Figura 4b, e a Figura 4c registra crescimento até o pico de 2.381 ignições no passo 99. Nas três cargas, o aumento das ignições decorre da expansão das frentes, e não de divergência entre as implementações.

6.2 Tempo, *speedup* e eficiência

A Tabela 3 apresenta as médias das cinco execuções medidas com -03. O valor após $\pm$ é a semi-amplitude do intervalo de confiança de 95%. As Figuras 5 e 6 apresentam os tempos e as métricas derivadas.

Tabela 3: Tempos médios com -O3 e IC 95%, em segundos.

Configuração	Pequena	Média	Grande
Sequencial	$0,328 \pm 0,004$	$3,784 \pm 0,034$	$13,101 \pm 0,008$
OpenMP, 1 thread	$0,349 \pm 0,005$	$3,978 \pm 0,019$	$13,833 \pm 0,035$
OpenMP, 2 threads	$0,179 \pm 0,007$	$2,063 \pm 0,044$	$7,107 \pm 0,163$
OpenMP, 4 threads	$0,097 \pm 0,006$	$1,100 \pm 0,042$	$3,771 \pm 0,027$
OpenMP, 8 threads	$0,084 \pm 0,006$	$0,908 \pm 0,003$	$3,152 \pm 0,007$
OpenMP, 16 threads	$0,114 \pm 0,002$	$0,969 \pm 0,044$	$3,195 \pm 0,018$

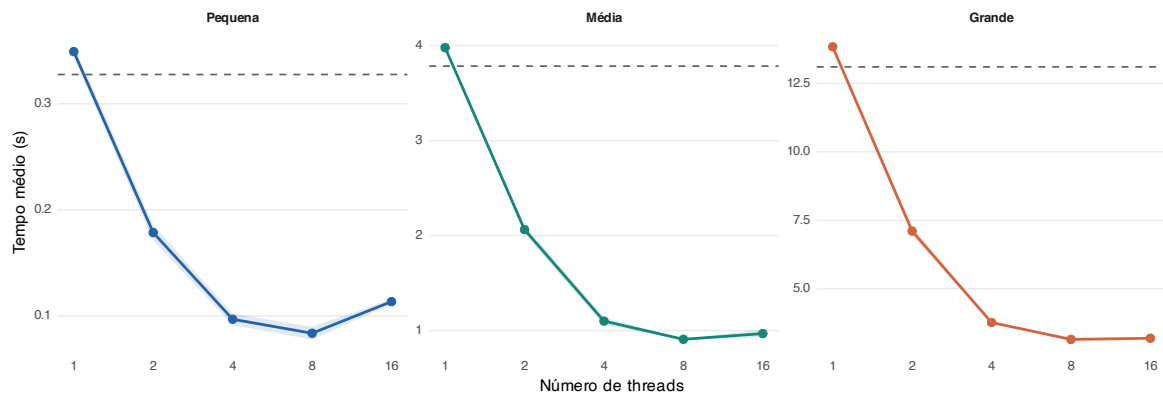
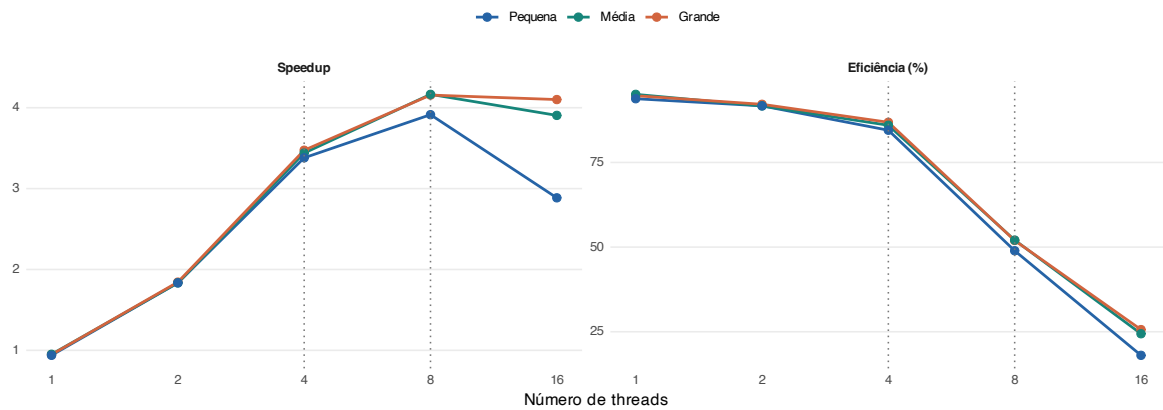
**Figura 5:** Tempo médio por número de threads. Cada painel preserva a escala da carga; as faixas indicam IC 95% e a linha tracejada representa a média sequencial.**Figura 6:** *Speedup* e eficiência em -O3. As linhas verticais marcam os quatro núcleos físicos e as oito CPUs lógicas.

Tabela 4: *Speedup* $S(T) = t_{seq}/t_{omp}(T)$ e eficiência $E(T) = S(T)/T$ por número de threads, com `static` e `-O3`.

Threads	<i>Speedup</i>			Eficiência (%)		
	Pequena	Média	Grande	Pequena	Média	Grande
1	0,94	0,95	0,95	93,8	95,1	94,7
2	1,83	1,83	1,84	91,7	91,7	92,2
4	3,38	3,44	3,47	84,5	86,0	86,9
8	3,91	4,17	4,16	48,9	52,1	52,0
16	2,89	3,91	4,10	18,0	24,4	25,6

O menor tempo ocorreu com 8 threads nas três cargas (Tabela 4). Os *speedups* foram 3,91, 4,17 e 4,16 para as entradas pequena, média e grande, com eficiência caindo de cerca de 86% em 4 threads para cerca de 52% em 8. Com 16 threads, os tempos aumentaram 35,6%, 6,7% e 1,4%, respectivamente, confirmando a saturação após as oito CPUs lógicas.

6.3 Comparação de *schedules*

Os *schedules* foram comparados na entrada média com 4 threads, correspondentes aos quatro núcleos físicos. Cada alternativa foi aplicada às construções com `schedule (runtime)`. Os valores exatos estão na Tabela 5.

Tabela 5: Comparação de *schedules* com 4 threads e `-O3`.

<i>Schedule</i>	Bloco	Tempo médio $\pm$ IC 95% (s)
<code>guided</code>	16	$1,079 \pm 0,014$
<code>static</code>	padrão	$1,100 \pm 0,042$
<code>dynamic</code>	64	$1,257 \pm 0,006$
<code>static</code>	16	$1,408 \pm 0,014$
<code>dynamic</code>	16	$2,000 \pm 0,003$

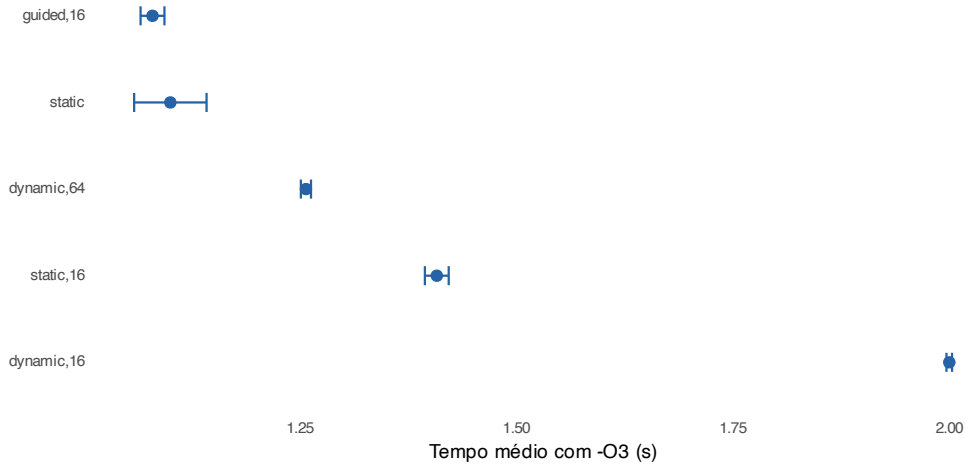


Figura 7: Média e IC 95% por *schedule*, na carga média com 4 threads. Menores valores indicam melhor desempenho.

6.4 Efeito de -O3 e das posições SIMD

A comparação principal entre -O2 e -O3 usou medições intercaladas na mesma carga, no mesmo nó e no mesmo período, o que isola o efeito da *flag*. Com 1, 4 e 8 threads, -O3 reduziu o tempo em 18,6%, 19,0% e 20,7%; o conjunto completo de medições confirmou reduções próximas de 20% nas três cargas. A Figura 8 mostra o fator t_{O2}/t_{O3} ; valores acima de 1 favorecem -O3.

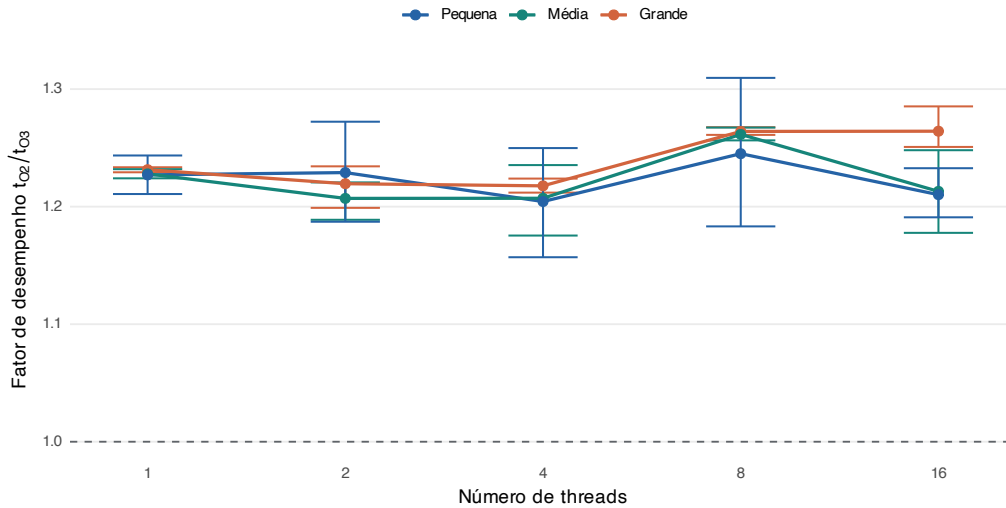


Figura 8: Fator de desempenho t_{O2}/t_{O3} por número de threads, com IC 95%. Valores acima de 1 indicam que -O3 é mais rápido; a redução de tempo fica próxima de 20% nas três cargas.

O ganho de tempo absoluto não implica aumento equivalente do *speedup*: -O3 acelerou as versões sequencial e OpenMP. Na carga média com 8 threads, por exemplo, o *speedup* passou de 4,14 para 4,17. Portanto, a flag melhora a execução final sem mudar materialmente a conclusão sobre escalabilidade.

Tabela 6: Penalidade das posições SIMD candidatas em relação à versão final, em -03.

Candidata	1 thread	4 threads	8 threads
Ativação de contenções	+2,21%	+1,40%	+1,86%
Atualização principal	+2,58%	+2,40%	+1,83%
Ambas	+3,48%	+3,02%	+4,35%

As três candidatas foram validadas funcionalmente e nenhuma melhorou o desempenho. Assim, adicionar `simd` onde o acesso e o controle são mais complexos seria uma regressão mensurável, apesar de a diretiva ser semanticamente permitida.

7. Análise dos Resultados

Esta seção relaciona as medições à arquitetura do nó, ao custo de escalonamento e às decisões de vetorização.

7.1 Escalabilidade e gargalos

Com uma thread, a versão OpenMP foi de 5,1% a 6,6% mais lenta que a sequencial devido à criação da equipe persistente, às reduções e às barreiras. Com quatro threads, os *speedups* foram 3,38, 3,44 e 3,47; com oito, chegaram a 3,91, 4,17 e 4,16. Na carga grande, a eficiência foi 92,2% com duas threads, 86,9% com quatro, 52,0% com oito e 25,6% com 16.

Até quatro threads, cada trabalhador pode ocupar um núcleo físico. Entre quatro e oito, o ganho adicional vem do *simultaneous multithreading* e é menor. Com 16 threads há sobresubscrição. Barreiras entre etapas, tráfego de memória e o trecho **single** também limitam a escalabilidade. A geração da floresta e o *checksum* são seriais, mas estão fora do intervalo cronometrado conforme a especificação.

7.2 Fração serial efetiva (Karp–Flatt)

Para separar o código intrinsecamente serial do custo que cresce com o número de threads, a Tabela 7 traz a fração serial de Karp–Flatt, $e(T) = (1/S(T) - 1/T)/(1 - 1/T)$, calculada a partir dos *speedups* medidos. Até quatro threads a fração é baixa (5–9%), condizente com boa escalabilidade nos núcleos físicos; a partir daí ela cresce (13–15% em oito threads e 19–30% em dezesseis), indicando que a saturação em torno de 4× decorre de sobrecusto crescente — SMT e sobresubscrição — e não de uma parte serial fixa.

Tabela 7: Fração serial de Karp–Flatt $e(T)$ (%) por número de threads, com **static** e -O3.

Threads	Pequena	Média	Grande
2	9,0	9,1	8,5
4	6,1	5,4	5,0
8	14,9	13,2	13,2
16	30,3	20,6	19,3

7.3 Justificativa do *schedule* escolhido

Guided,16 e **static** produziram 1,079 s e 1,100 s, com intervalos sobrepostos. Essa evidência não sustenta declarar uma política superior. **Dynamic**,64, **static**,16 e **dynamic**,16 foram aproximadamente 14,2%, 28,0% e 81,8% mais lentos que **static**. A implementação mantém **static** como padrão por sua distribuição previsível e baixo custo, e conserva **schedule(runtime)** para permitir reprodução das comparações.

7.4 Interpretação da otimização e do SIMD

Os resultados distinguem três efeitos. Primeiro, a paralelização reduz o tempo a partir de duas threads. Segundo, `-O3` reduz o tempo de ambas as versões em cerca de 20%, preservando o *speedup*. Terceiro, uma diretiva SIMD só ajuda quando o compilador encontra uma forma vetorial eficiente; inseri-la em laços com controle complexo não garante ganho. A decisão final combina diagnóstico do GCC, equivalência funcional e medição controlada.

8. Conclusão

As versões sequencial e OpenMP produziram resultados funcionais idênticos nos 80 casos de teste da referência independente, nas três entradas oficiais e nas diferentes configurações de threads e *schedules*. A validação reuniu 1.280 execuções válidas sem divergência e 36 rejeições corretas de entradas inválidas.

A configuração mais rápida foi `static` com 8 threads. Os melhores *speedups* foram 3,91, 4,17 e 4,16 nas cargas pequena, média e grande. A troca para `-O3` reduziu o tempo absoluto em aproximadamente 20%, enquanto a sobresubscrição com 16 threads piorou todas as cargas. As diretivas SIMD permaneceram apenas nas duas reduções lineares: os locais adicionais foram corretos, mas 1,4% a 4,4% mais lentos.

Os resultados mostram que a melhor versão não decorre da soma indiscriminada de diretivas. Ela combina uma região paralela persistente, divisão de trabalho regular, reduções, sincronização explícita no desenho e otimizações aceitas somente após validação e medição reproduzível.